CSC718 – Assignment 2 Scaling Study and Report

Joshua Ganschow

Prof. Abid Mehmood

10.25.26

Master-Worker Design and Message Protocol Description

For section R1, I used a simple round-robin allocation of chunks. Starting with worker 1 getting range $[2, \min(2 + CHUNK - 1, N)]$ and setting the new start to $2 + CHUNK$, we continue with worker 2 getting range $[start, \min(start + CHUNK - 1, N)]$ and so on. Using a circular round robin, we loop back around to worker 1 after assigning a segment to the last worker. These segments are stored in a vector of pairs for each of the workers and each worker is passed the entire vector of their assigned segments at the end. This only requires two types of messages: *WORK* and *RESULT*. The former being the message being sent to the worker and the latter being the type sent back to the master.

For section R2, I used the assigned request model. I maintain where the last segment ended (this starts at 2) and for each request that the master worker receives it will either give the next segment (sent with a *WORK* tag) if there are still segments to give or will give a poison job to tell the worker to stop. This loop keeps going until we've sent a job to every worker. The *STOP* job is only sent when the worker requests new work but there is none left to give. The master worker can receive a *REQUEST* for more work (one type of tag) or a *RESULT* of work done (the final type of tag). The bookkeeping of maintaining the current start of the next chunk is the exact same as the logic for section R1.

Correctness Evidence

Ran the program with debug on which forces the main worker (0) to calculate the number of primes sequentially and print out if the parallel version was correct at the end. The `correct? YES` line is that check.

Running command `mpirun -np=8 ./r1 200000 2000` yields the following output. Only the first few lines and last few lines are shown for brevity. I ran this on my home computer, but the output is the same if run on the Rushmore cluster.

```
joshua@fedora:~/school/Fall 2026/CSC718/content/module 4 - MPI Programming/ass
ignment2$ mpirun -np=8 ./r1 200000 2000
Rank: 0 Processes: 8    Hostname: fedora
master sending [2...2001] to worker 1
master sending [14002...16001] to worker 1
master sending [28002...30001] to worker 1
Rank: 1 Processes: 8    Hostname: fedora
Rank: 7 Processes: 8    Hostname: fedora
Rank: 4 Processes: 8    Hostname: fedora
Rank: 3 Processes: 8    Hostname: fedora
```

```
master gathered 2516 from worker 3
master gathered 2498 from worker 4
master gathered 2517 from worker 5
master gathered 2533 from worker 6
master gathered 2422 from worker 7
number of primes in range [2,200000] = 17984
expected? YES
Time taken: 0.0265457 seconds
N = 200000, CHUNK = 2000, P = 8
```

Static vs. Dynamic Comparison

Using values N=50,000,000 and CHUNK=50,000 with P=4. Averaging the time for three runs.

Running on the Rushmore cluster.

Version	Time (s)
Static	29.9852
Dynamic	47.5535

Dynamic was much slower here. This is not very surprising. Since each segment (length CHUNK) given for work is about the same amount of computation for the process, we most likely don't see certain processors being idle while others are still working. The "requesting work" model works very well if we have the scenario where the amount of work associated with a segment is not balanced between segments. Counting primes is not this type of work.

Timing Tables

Using the same values N=50,000,000 and CHUNK=50,000. Taking three measurements and averaging them for each of the P={4,8,16}. After testing with different values of CHUNK, it didn't seem to make much of a difference; as long as the segments were small enough that each worker got about the same amount of work (i.e. CHUNK ≤ N/P)

P	Time (s)	Speedup: T(4)/T(P)	Efficiency: Speedup/(P/4)
4	41.8302	1	1
8	49.977	0.8370	0.4185
16	46.3893	0.9017	0.2254

Now with N=100,000,000 and CHUNK=100,000

P	Time (s)	Speedup: T(4)/T(P)	Efficiency: Speedup/(P/4)
4	111.909	1	1
8	126.636	0.8837	0.4419
16	103.528	1.0810	0.2702

Execution Evidence

1. P=4
   a.
```
mpiuser@RushmoreNFS:~/nfs/CSC718/openMPI/assignment2$ mpirun -np=4 ---hos
tfile machinefile.dsu ./dyn 50000000 50000
Rank: 0 Processes: 4    Hostname: Lincoln
Rank: 1 Processes: 4    Hostname: Washington
Rank: 3 Processes: 4    Hostname: Jefferson
Rank: 2 Processes: 4    Hostname: Roosevelt
number of primes in range [2,50000000] = 3001134
Time taken: 43.5084 seconds
N = 50000000, CHUNK = 50000, P = 4
```
2. P=8
   a.
```
mpiuser@RushmoreNFS:~/nfs/CSC718/openMPI/assignment2$ mpirun -np=8 ---hos
tfile machinefile.dsu ./dyn 50000000 50000
Rank: 6 Processes: 8    Hostname: Roosevelt
Rank: 2 Processes: 8    Hostname: Roosevelt
Rank: 1 Processes: 8    Hostname: Washington
Rank: 5 Processes: 8    Hostname: Washington
Rank: 4 Processes: 8    Hostname: Lincoln
Rank: 7 Processes: 8    Hostname: Jefferson
Rank: 0 Processes: 8    Hostname: Lincoln
Rank: 3 Processes: 8    Hostname: Jefferson
number of primes in range [2,50000000] = 3001134
Time taken: 50.6145 seconds
N = 50000000, CHUNK = 50000, P = 8
```
3. P=16
   a.
```
mpiuser@RushmoreNFS:~/nfs/CSC718/openMPI/assignment2$ mpirun -np=16 ---ho
stfile machinefile.dsu ./dyn 50000000 50000
Rank: 12        Processes: 16   Hostname: Lincoln
Rank: 0 Processes: 16   Hostname: Lincoln
Rank: 3 Processes: 16   Hostname: Jefferson
Rank: 4 Processes: 16   Hostname: Lincoln
Rank: 14        Processes: 16   Hostname: Roosevelt
Rank: 15        Processes: 16   Hostname: Jefferson
Rank: 10        Processes: 16   Hostname: Roosevelt
Rank: 1 Processes: 16   Hostname: Washington
Rank: 9 Processes: 16   Hostname: Washington
Rank: 5 Processes: 16   Hostname: Washington
Rank: 8 Processes: 16   Hostname: Lincoln
Rank: 2 Processes: 16   Hostname: Roosevelt
Rank: 7 Processes: 16   Hostname: Jefferson
Rank: 13        Processes: 16   Hostname: Washington
Rank: 6 Processes: 16   Hostname: Roosevelt
Rank: 11        Processes: 16   Hostname: Jefferson
number of primes in range [2,50000000] = 3001134
Time taken: 50.0801 seconds
N = 50000000, CHUNK = 50000, P = 16
```

Interpretation

The output confused me at first until I did some investigating into the stats of the Rushmore cluster. Each of the four nodes has 1 vCPU. Predictably, this means that increasing MPI processes-per-node results in no speed gain; it actually results in a speed loss! This is most likely because of the processes each contending for a share of the CPU time. For just one process on each, the CPU can work through without needing to context switch beyond normal OS usage. For multiple processes, it, of course, needs to.

If we had multiple processors per node, we would expect to see a more “ideal” speedup near 2x and 4x for 8 and 16 processes respectively. We would not get a perfectly ideal speedup because there is more communication overhead associated with more processes. More messages need to be passed between master and workers (and vice-versa). Running the program on my computer that has 8 cores, I see a predictable speedup of about 1.9x moving from 4 to 8 processes.